

OneNexium

SPRINT ZERO — REVISED 10-DAY PLAN

AI-Accelerated with Claude Code + Cursor · Full Feature Set · 10 Days

10 Days Full Sprint	Claude Code AI Dev	Cursor AI Dev	Full Stack In + Backend	Live Product Day 10 Goal
-------------------------------	------------------------------	-------------------------	-----------------------------------	------------------------------------

What changed from v1

This is the REVISED plan. Scope expanded because Claude Code + Cursor makes engineers 3-5x faster on boilerplate. More features are now achievable in 10 days.

The non-negotiable rule

Quality still does not matter. Connectivity and completeness of the feature set is the goal. Fix quality in Sprint 1.

The Day 10 definition

A non-technical person signs up, onboards, generates a 4-page site with backend, connects a domain, and sees it live. One person. 10 days.

01 WHY CLAUDE CODE + CURSOR CHANGES EVERYTHING

The AI development acceleration that makes this expanded scope possible in 10 days

"With Claude Code and Cursor, two engineers move at the speed of a five-person team. Every scaffold, every CRUD route, every component — generated in seconds, not hours."

THE SPEED MULTIPLIER — WHAT CHANGES

Task Type	Manual Speed
Next.js page scaffold (layout, components, types)	3 hours
Drizzle ORM schema + migrations	2 hours
API routes (CRUD — GET/POST/PATCH/DELETE)	4 hours
NextAuth configuration (providers, callbacks, middleware)	3 hours
RBAC middleware + role guards	2 hours
Responsive Tailwind + shadcn/ui page	4 hours
MCP server tool (FastMCP function)	1 hour
Custom domain DNS + SSL logic	3 hours
Onboarding flow (multi-step form)	3 hours
Self-healing retry loop	2 hours

HOW TO USE CLAUDE CODE EFFECTIVELY IN THIS SPRINT

 Spec first, generate second — Write a 5-line spec of what you need before running Claude Code.
Bad: "build auth" Good: "NextAuth with Google + email magic link, session in JWT, middleware protects /app/* routes, role stored in JWT as user.role"

 Use Claude Code for scaffolding, Cursor for iteration — Claude Code for first-pass generation, Cursor's tab completion and inline edits for refinements and bug fixes.

 Test immediately after generation — Do not generate 10 files then test. Generate 1 file, test, generate next.

- 📁 Commit every working state — After each task that passes acceptance criteria, git commit. If the next task breaks something, you can roll back in 10 seconds.
- 🎯 One task at a time — Do not ask Claude Code to "build the entire auth system". Ask it to "write the NextAuth configuration file with Google and email providers".
- 🔗 Use Cursor for MCP tools — MCP tool functions are short and repetitive. Cursor autocompletes the pattern after the first one. Engineer 1 writes the second tool in 5 minutes by tab-completing.

REVISED SCOPE — WHAT IS NOW IN

✅ IN SCOPE — REVISED	❌ STILL OUT OF SCOPE
✅ Platform EC2 + Runtime EC2 on AWS	❌ Billing and Stripe integration (Sprint 1)
✅ Docker + Traefik + wildcard SSL	❌ Visual overlay editor (Sprint 2)
✅ RDS PostgreSQL + Drizzle ORM — platform schema	❌ GoHighLevel-style CRM features (later)
✅ MCP /codegen — write, read, list, apply_diff	❌ Multi-tenant infrastructure (full Neon/Upstash per user)
✅ MCP /build — trigger, errors, self-healing retry	❌ Full Langfuse observability (Sprint 1)
✅ MCP /deploy — Docker build, container, Traefik route	❌ K8s / ECS migration (later)
✅ Backend code generation — Drizzle schema + API routes	❌ Marketplace of integrations (later)
✅ Multi-page generation — Home, About, Services, Contact	❌ Advanced analytics dashboard (Sprint 2)
✅ Beautiful UI — proper Tailwind + shadcn/ui design	❌ Mobile app (Phase 3)
✅ Self-healing build retry loop (3 attempts)	❌ Figma import (later)
✅ Authentication — NextAuth (Google + email magic link)	❌ White-label agency platform (Sprint 4)
✅ RBAC — Admin / Member / Viewer roles	❌ Advanced AI model routing (Sprint 2)
✅ Onboarding flow — 4-step wizard	❌ Team collaboration features (Sprint 2)
✅ Custom domain — DNS instructions + polling + SSL	❌ Semantic caching (Sprint 2)
✅ Mobile responsive platform UI	❌ Advanced Lighthouse optimisation (Sprint 2)
✅ Live preview iframe	❌ Real-time push notifications (Sprint 2)
✅ Project persistence and conversation history	❌ Custom component library (Sprint 3)

02 PRE-SPRINT SETUP

Everything installed and configured BEFORE Day 1 — zero day for accounts and tools

ACCOUNTS TO CREATE THIS WEEK

Service	Action
AWS	Root account, MFA, IAM user onenexium-dev, AdministratorAccess policy, generate access keys
Anthropic	API key at console.anthropic.com. \$500 limit for sprint. Store as ANTHROPIC_API_KEY.
GitHub	Org "onenexium". Repos: onenexium-platform + onenexium-mcp. Both engineers as admins.
Cloudflare	Free account. Add domain onenexium.com. Wildcard DNS: *.sites.onenexium.com → Runtime EC2.
Claude Code	Install: npm install -g @anthropic-ai/claude-code. Authenticate with Anthropic API key. Both engineers.
Cursor	Install at cursor.com. Both engineers. Connect to GitHub repos. Set Claude as AI model.
Vercel (optional)	For platform hosting in MVP — faster than EC2 for Next.js. Decide: Vercel or EC2.

AWS RESOURCES — PROVISION BEFORE DAY 1

```
# EC2 Instances
Platform EC2:  t3.large  (2 vCPU, 8GB)   — Amazon Linux 2023
Runtime EC2:   t3.xlarge (4 vCPU, 16GB)  — Amazon Linux 2023

# RDS
Engine: PostgreSQL 16, Instance: db.t3.micro
DB: onenexium_platform, User: onenexium_admin

# ECR
Repository: onenexium/user-apps

# Secrets Manager
onenexium/platform/DATABASE_URL
onenexium/platform/ANTHROPIC_API_KEY
onenexium/platform/MCP_AUTH_TOKEN  (random 64-char string)
onenexium/platform/NEXTAUTH_SECRET (random 64-char string)

# Security Groups
platform-sg: 22, 80, 443, 3000
runtime-sg:  22, 80, 443, 3001-3999
rds-sg:      5432 from platform-sg only
```

SOFTWARE STACK — BOTH ENGINEERS INSTALL

Tool	Purpose	Install
Node.js 20 LTS	Platform + MCP development	nvm install 20
Python 3.12	MCP server (FastMCP)	pyenv install 3.12
Docker Desktop	Local container testing	docs.docker.com
AWS CLI v2	EC2, ECR, Secrets Manager	brew install awscli
Claude Code CLI	AI pair programming — primary dev tool	npm i -g @anthropic-ai/claude-code
Cursor IDE	AI-powered code editor — secondary tool	cursor.com
pnpm	Faster npm for Next.js platform	npm i -g pnpm
Drizzle Kit	Database migrations CLI	pnpm add -D drizzle-kit

03 FEATURE SPECIFICATIONS

Exact spec for each in-scope feature — give these to Claude Code verbatim

Before running Claude Code on any feature, use these specs as your prompt. The more specific your spec, the better the output.

FEATURE 1 — AUTHENTICATION (NEXTAUTH + RBAC)

SPEC FOR CLAUDE CODE:

Build NextAuth v5 (Auth.js) with:

- Providers: Google OAuth + Email magic link (Resend)
- Session: JWT strategy, 24hr expiry
- Callbacks: `jwt()` adds `user.id` and `user.role` to token
- `session()` exposes `user.id` and `user.role` to client

Database: Drizzle ORM PostgreSQL

- Tables: `users`, `accounts`, `sessions`, `verification_tokens` (NextAuth schema)
- Add: role ENUM (`admin`, `member`, `viewer`) to `users` table, default `"member"`

Middleware (`middleware.ts`):

- Protect: `/app/*` routes — redirect to `/login` if no session
- Public: `/login`, `/signup`, `/onboarding`, `/`, `/api/auth/*`

RBAC guards (`lib/auth/rbac.ts`):

- `requireRole(role: Role)` — HOC for server components
- `useRole()` — client hook
- Roles: `admin` (`all`), `member` (`create/edit own`), `viewer` (`read only`)

Files to generate:

- `auth.ts` — NextAuth config
- `middleware.ts` — route protection
- `app/login/page.tsx` — clean login UI
- `lib/auth/rbac.ts` — role utilities
- `lib/db/schema/auth.ts` — Drizzle auth tables

FEATURE 2 — ONBOARDING FLOW

SPEC FOR CLAUDE CODE:

Build a 4-step onboarding wizard at `/onboarding`

Triggered on first login (check `user.onboarded === false` in session)

Step 1 — Business Type

- Title: "What kind of business do you run?"
- Options (cards with icons): `Coaching` / `Consulting` / `Agency` / `Freelance` / `Other`
- Persist: `workspace.industry` = selected type

Step 2 — Business Name + Description

- Inputs: Business name (text), Short description (textarea, 2-3 sentences)
- Persist: `workspace.name`, `workspace.description`

Step 3 – Branding

Primary colour picker (simple 8-colour palette + hex input)
Logo upload (image file, stored in S3)
Font preference (3 options: Modern / Classic / Playful)
Persist: workspace.brand_color, workspace.logo_url, workspace.font_style

Step 4 – First Project Setup

"What do you want to build first?"
Pre-fills the AI chat with: "Build a website for {business_name}..."
Button: "Start Building" → redirects to /app with pre-filled prompt

Components: stepper progress bar, animated transitions between steps
On complete: mark user.onboarded = true, create workspace record in DB

FEATURE 3 — MULTI-PAGE GENERATION

SPEC FOR CLAUDE CODE (system prompt addition):

When generating a website, always create these files:

app/layout.tsx	– Root layout: navbar, footer, font setup
app/page.tsx	– Homepage: hero, features/services, CTA, testimonials
app/about/page.tsx	– About: story, team, mission, values
app/services/page.tsx	– Services: service cards with descriptions + pricing
app/contact/page.tsx	– Contact: form (name, email, message), address, social links
components/Navbar.tsx	– Responsive nav with mobile hamburger menu
components/Footer.tsx	– Footer with links, social icons, copyright
components/ui/*	– Import only from shadcn/ui – do not create custom primitives

Writing order: layout.tsx FIRST, then page.tsx, then sub-pages

Every page must: import { Metadata } from "next" and export const metadata

Every page must: be mobile-responsive (Tailwind responsive prefixes: sm: md: lg:)

Colour scheme: use the CSS variables --primary, --secondary from workspace brand config

FEATURE 4 — BACKEND CODE GENERATION (DRIZZLE + API ROUTES)

SPEC FOR CLAUDE CODE (backend generation phase):

After frontend is approved, detect required backend features and generate:

Drizzle ORM Schema (lib/db/schema/{feature}.ts):

Contact form → contacts table: id, name, email, message, created_at
Booking → bookings table: id, service_id, customer_email, slot_start, slot_end, status
Newsletter → subscribers table: id, email, confirmed, subscribed_at

API Routes (app/api/{feature}/route.ts):

POST /api/contact → validate with Zod → insert to contacts → send email via Resend
GET /api/bookings → fetch bookings for authenticated user
POST /api/bookings → validate slot availability → insert booking → confirmation email
POST /api/subscribe → validate email → insert subscriber → welcome email

```
Every API route must:  
  Use Zod for request validation (z.object({...}).parse(body))  
  Return proper HTTP status codes (200, 201, 400, 401, 500)  
  Be typed: export async function POST(request: Request): Promise<Response>  
  Handle errors: try/catch with NextResponse.json({ error: message }, { status: 500 })
```

```
Migration: generate and run Drizzle migration after schema generation  
npx drizzle-kit generate → npx drizzle-kit migrate
```

FEATURE 5 — SELF-HEALING BUILD RETRY LOOP

SPEC FOR CLAUDE CODE:

Build a retry loop in the MCP /build namespace:

```
async function buildWithHealing(projectId: string, maxRetries = 3) {  
  for (let attempt = 1; attempt <= maxRetries; attempt++) {  
    const result = await triggerBuild(projectId)  
    if (result.status === "success") return { success: true, attempts: attempt }  
  
    // Feed errors back to Claude for auto-fix  
    const errors = await getBuildErrors(result.jobId)  
    const formattedErrors = formatErrors(errors) // group by file  
  
    await callClaude({  
      role: "user",  
      content: `Build failed (attempt ${attempt}/${maxRetries}).  
Fix these TypeScript errors:\n${formattedErrors}  
Rules: do not change logic, only fix type errors.  
Use write_file to save each fixed file.`  
    })  
  }  
  return { success: false, message: "Restored previous version" }  
}  
  
// Chat UI shows during retry:  
// "Build failed – fixing TypeScript errors automatically... (attempt 2/3)"
```

FEATURE 6 — CUSTOM DOMAIN CONNECTION

SPEC FOR CLAUDE CODE:

Build domain connection flow at /app/projects/{id}/domain

Step 1 – User enters domain (e.g. app.theirbusiness.com)
Validate: is it a valid domain format?
WHOIS lookup (or manual registrar detection from domain extension)
Show DNS instructions specific to their registrar:
 Type: CNAME
 Name: app (or @ for root domain)
 Value: {tenant_id}.sites.onenexium.com
 TTL: 300

Step 2 – DNS polling


```
API route: POST /api/domains/verify
Every 60 seconds: dns.resolve(domain, "CNAME") → check if value matches
Store in DB: domains.status = pending | verified | failed
Show progress: animated "Waiting for DNS..." with time estimate
```

Step 3 — On DNS verified:

```
Call /deploy/register_traefik_route with custom domain
Traefik adds routing rule + issues Let's Encrypt cert
Update DB: domains.ssl_active = true
Email user: "Your domain is live!"
```

DB schema (Drizzle):

```
domains: id, project_id, domain, status, ssl_active, verified_at, created_at
```

FEATURE 7 — BEAUTIFUL UI (PLATFORM DESIGN SYSTEM)

SPEC FOR CURSOR (not Claude Code — iterative design work):

Design system built on: Tailwind CSS + shadcn/ui + Lucide icons

Colour palette:

```
Background: #0F172A (dark mode) or #F8FAFC (light mode)
Primary: #4F46E5 (indigo)
Accent: #7C3AED (violet)
Surface: #1E293B (cards in dark mode)
```

Layout:

```
Left sidebar: 240px, collapsible to 60px (icon-only mode)
Sidebar items: icon + label, active state with primary colour bg
Top bar: breadcrumb left, user avatar + notifications right
Main: max-w-7xl, centered, padding 24px
```

Chat interface:

```
Messages: user (right, indigo bubble) + AI (left, surface card)
Tool use chips: small badges showing "Writing files..." "Building..." etc.
Input: fixed bottom, multi-line textarea, send button, file upload icon
Preview: split-pane — chat left (40%), iframe right (60%)
```

Responsive:

```
Mobile: sidebar hidden → hamburger menu → slide-over drawer
Tablet: sidebar collapsed to icons
Desktop: full sidebar expanded
```

04 THE REVISED 10-DAY EXECUTION PLAN

Day-by-day plan with Claude Code prompts, tasks, owners, and acceptance criteria

"With AI-assisted development, tasks that took hours now take minutes. The plan below reflects Claude Code and Cursor acceleration throughout."

PHASE 1 DAYS 1 – 2

Infrastructure + Auth + Onboarding

DAY

1

Infrastructure + Traefik + Auth Scaffold

EC2 up, Docker + Traefik running, Next.js platform created, NextAuth scaffold generated with Claude Code

T1.1 AWS Infrastructure Setup ⌚ 2 hrs 👤 Engineer 1

- Launch Platform EC2 (t3.large) and Runtime EC2 (t3.xlarge) — Amazon Linux 2023
- SSH both instances — install on each: git, nvm (Node 20), Python 3.12, Docker, Docker Compose, AWS CLI
- Platform EC2: attach IAM role with Secrets Manager + ECR access
- Create /data/projects/, /data/build-sandbox/, /data/templates/, /data/build-logs/ directories
- Create Security Groups as specced — platform-sg, runtime-sg, rds-sg
- Confirm RDS is reachable: psql from Platform EC2 → CREATE DATABASE onenexium_platform
- Create ECR repo: onenexium/user-apps — note full URI
- Configure aws cli on both EC2s: aws configure with onenexium-dev credentials

⚡ **Claude Code / Cursor:** Use AWS CLI to launch EC2s. Ask Claude Code: "Write me AWS CLI commands to launch two EC2 instances with correct security groups and IAM roles for a Next.js + Docker platform."

T1.2 Docker + Traefik on Runtime EC2 ⌚ 2 hrs 👤 Engineer 1

- Install Docker Engine on Runtime EC2 (not Docker Desktop)
- Create Docker network: docker network create onenexium-runtime
- Create /data/traefik/ directory structure
- Write traefik.yml: entrypoints (web:80, websecure:443), certificatesResolvers (Let's Encrypt httpChallenge), api.insecure=false
- Write docker-compose.yml for Traefik: mount /var/run/docker.sock, volumes for Let's Encrypt certs, network onenexium-runtime
- Add HTTP→HTTPS redirect middleware in Traefik config
- docker compose up -d — confirm Traefik running: docker ps
- Update Cloudflare DNS: *.sites.onenexium.com CNAME → Runtime EC2 public IP
- Test: deploy nginx test container with Traefik labels → https://hello.sites.onenexium.com → SSL padlock

⚡ **Claude Code / Cursor:** Ask Claude Code: "Write a complete Traefik v3 docker-compose.yml with Let's Encrypt ACME http-01 challenge, Docker provider, automatic HTTPS redirect, and wildcard subdomain support."

T1.3 Next.js Platform — Project Setup ⌚ 1 hr 👤 Engineer 2

- `pnpm create next-app@latest onenexium-platform --typescript --tailwind --app --src-dir`
- Install dependencies: `pnpm add drizzle-orm postgres @auth/drizzle-adapter next-auth @anthropic-ai/sdk lucide-react`
- Install shadcn/ui: `pnpm dlx shadcn@latest init` (choose Slate base colour, CSS variables yes)
- Add shadcn components: button, input, textarea, card, badge, avatar, dropdown-menu, sheet, toast, progress
- Create `.env.local` with all required variables (see T1.4)
- Push to GitHub: onenexium-platform main branch
- Set up GitHub Actions: `.github/workflows/deploy.yml` — push to main → deploy to Platform EC2 via SSH + PM2

⚡ **Claude Code / Cursor:** After create-next-app: open in Cursor. Ask: "Set up this Next.js project with Drizzle ORM (PostgreSQL), NextAuth v5, and a dark sidebar layout with shadcn/ui."

T1.4 NextAuth + Drizzle Auth Schema ⌚ 3 hrs 💎 Engineer 2

- Run Claude Code with Feature 1 spec from Section 03
- Claude Code generates: `auth.ts`, `middleware.ts`, `app/login/page.tsx`, `lib/auth/rbac.ts`
- Drizzle schema: `lib/db/schema/auth.ts` — users, accounts, sessions, verification_tokens tables
- Add role column to users: `role: text("role").default("member").$type<"admin"|"member"|"viewer">()`
- Set up Drizzle config: `drizzle.config.ts` pointing to RDS
- Run first migration: `pnpm drizzle-kit generate` && `pnpm drizzle-kit migrate`
- Test: `npm run dev` → navigate to `/login` → Google login button visible
- Configure Google OAuth: add Google client ID + secret to `.env.local`
- Test complete auth flow: login with Google → session created → `/app` redirects to `/login` if not authed

⚡ **Claude Code / Cursor:** Claude Code prompt: "Using the spec below, implement NextAuth v5 with Google OAuth and email magic link, Drizzle PostgreSQL adapter, JWT sessions with user role, middleware protecting `/app/*` routes, and RBAC utilities." [paste Feature 1 spec]

✅ END OF DAY ACCEPTANCE CRITERIA

1. `https://hello.sites.onenexium.com` loads with SSL (Traefik routing works)
2. `npm run dev` shows Next.js platform at `localhost:3000`
3. Navigating to `/app` redirects to `/login` (middleware working)
4. Google OAuth login completes — user row created in RDS users table
5. Drizzle migration ran — all auth tables exist in PostgreSQL
6. `/api/auth/session` returns session with `user.role` field

🎯 **MILESTONE: Auth pipe proven. Infrastructure live. Traefik routing SSL.**

Onboarding Flow + Platform Layout + MCP Foundation

4-step onboarding working, platform shell beautiful, MCP FastAPI server skeleton live

T2.1 4-Step Onboarding Flow ⌚ 3 hrs 👤 Engineer 2

- Run Claude Code with Feature 2 spec from Section 03
- Claude Code generates: app/onboarding/page.tsx with 4-step wizard component
- Create Drizzle schema: lib/db/schema/workspaces.ts
 - workspaces: id, owner_id, name, description, industry, brand_color, logo_url, font_style, created_at
 - projects: id, workspace_id, name, status, tenant_id, subdomain, created_at
 - ai_conversations: id, project_id, messages (jsonb), created_at
- Run migration: pnpm drizzle-kit generate && pnpm drizzle-kit migrate
- Onboarding redirect: after first login, check user.onboarded → redirect to /onboarding
- On wizard complete: create workspace record → mark user.onboarded = true → redirect to /app
- Test full flow: login → onboarding → complete → lands on /app

⚡ **Claude Code / Cursor:** After Claude Code generates the wizard: use Cursor to refine transitions and polish the stepper component. Ask Cursor: "Add smooth fade transitions between steps and a progress indicator showing 1/4, 2/4, 3/4, 4/4."

T2.2 Platform Shell Layout — Beautiful Sidebar + Responsive ⌚ 3 hrs 👤 Engineer 2

- Run Claude Code with Feature 7 spec from Section 03 (platform layout section)
- Generate: app/(app)/layout.tsx — sidebar + top bar wrapper
- Sidebar items: Dashboard, Projects, Domains, Settings
- Sidebar collapse: stores state in localStorage, icon-only mode on collapse
- Top bar: workspace name left, user avatar dropdown right (profile, sign out)
- Mobile: sidebar hidden by default, hamburger icon opens slide-over sheet (shadcn Sheet)
- All routes under app/(app)/ are protected by middleware automatically
- Apply to existing pages — test all breakpoints: mobile (375px), tablet (768px), desktop (1280px)
- Test: resize browser window — sidebar collapses, layout does not break

⚡ **Claude Code / Cursor:** Cursor prompt for responsive sidebar: "Make this sidebar collapse to icon-only mode on md screens and hide completely on mobile with a hamburger menu using shadcn Sheet component."

T2.3 MCP Server — FastMCP Foundation ⌚ 3 hrs 👤 Engineer 1

- cd /data/mcp-server/ → python3 -m venv venv → source venv/bin/activate
- pip install fastmcp fastapi uvicorn asyncpg boto3 python-dotenv paramiko
- Run Claude Code: "Build a FastMCP + FastAPI server with 5 route namespaces: /workspace, /codegen, /build, /infra, /deploy. Each namespace has a health check tool. JWT auth middleware validates Authorization header against MCP_AUTH_TOKEN env var. Structured logging to stdout."
- Verify all 5 namespaces are created with placeholder tools
- Run: uvicorn main:app --host 0.0.0.0 --port 8000
- Test: curl http://localhost:8000/health → {"status":"ok"}

- Install Caddy for HTTPS: install caddy on Platform EC2
- Caddyfile: `mcp.onenexium.com { reverse_proxy localhost:8000 }`
- Test: `curl https://mcp.onenexium.com/health` → 200 with SSL

⚡ **Claude Code / Cursor:** *Claude Code does the entire FastMCP structure in one shot with the right prompt. Key: specify all 5 namespaces, JWT auth pattern, and FastAPI lifespan for DB pool setup.*

T2.4 Base Next.js Template for Generated Apps ⌚ 1 hr 👤 Engineer 1

- Create `/data/templates/base-nextjs/` — minimal Next.js 14 App Router project
- Run: `claude-code "Create a minimal Next.js 14 App Router template with TypeScript strict, Tailwind CSS, shadcn/ui (slate), Drizzle ORM with postgres adapter, CSS variables for theming (--primary, --secondary, --background). Include: root layout, empty homepage, basic Navbar and Footer components."`
- Run: `npm install` & `npm run build` — confirm zero errors
- This is the scaffold every user project starts from
- Store Dockerfile in template: FROM node:20-alpine, WORKDIR /app, COPY, RUN npm install, RUN npm run build, EXPOSE 3000, CMD npm start

✅ END OF DAY ACCEPTANCE CRITERIA

1. Login → Onboarding (4 steps) → completes → redirects to /app correctly
2. Workspace record created in RDS after onboarding completion
3. Platform sidebar shows on all screen sizes correctly (mobile drawer, tablet icons, desktop full)
4. FastMCP server responds at `https://mcp.onenexium.com/health` with SSL
5. Base Next.js template builds clean: `npm run build` exits 0
6. All platform routes under /app require authentication

🎯 **MILESTONE: Onboarding, layout, and MCP foundation all working. Platform is usable.**

PHASE 2 DAYS 3 – 4

All MCP Tools Built + Chat UI Connected

DAY
3

All MCP Tools — /codegen, /build, /infra

Build all tools in codegen, build, and infra namespaces using Claude Code (rapid generation)

T3.1 MCP /codegen — 4 tools ⌚ 2 hrs 👤 Engineer 1

- Claude Code prompt: "In the /codegen FastMCP namespace, implement these 4 tools:",
- `write_file(project_id, file_path, content)` — writes to `/data/projects/{id}/{path}`, creates dirs
- `read_file(project_id, file_path)` — reads and returns file content
- `list_files(project_id)` — walks directory tree, returns relative paths array
- `apply_diff(project_id, file_path, old_str, new_str)` — targeted string replacement in file
- Each tool: validates `project_id` exists, handles `FileNotFoundError` gracefully
- Test all 4 with `curl` — including `apply_diff` on a real file

⚡ **Claude Code / Cursor:** Claude Code writes all 4 tools in under 10 minutes. After generation: use Cursor tab completion to add error handling variations. The pattern repeats.

T3.2 MCP /build — 3 tools + self-healing logic ⌚ 3 hrs 👤 Engineer 1

- Claude Code prompt: "In /build FastMCP namespace, implement:",
- `trigger_build(project_id)` — copies base template, overlays project files, runs `npm run build` in Docker sandbox, returns `{status, job_id, errors}`
- `get_build_errors(job_id)` — reads structured error log, returns `[{file, line, message}]`
- `get_preview_url(project_id)` — returns the current subdomain URL if deployed
- Claude Code prompt (self-healing): "Build a `build_with_healing(project_id)` function that retries up to 3 times, feeding TypeScript errors back to Claude API on each failure."
- Store build logs: `/data/build-logs/{job_id}.json`
- Test: write broken TypeScript → `trigger_build` → confirm errors returned → fix file → `trigger_build` again → success

⚡ **Claude Code / Cursor:** Self-healing loop: Claude Code generates the retry wrapper cleanly. Give it the Feature 5 spec from Section 03. It handles the Claude API call internally.

T3.3 MCP /infra — 4 tools ⌚ 2 hrs 👤 Engineer 1

- Claude Code prompt: "In /infra FastMCP namespace, implement:",
- `provision_tenant_schema(tenant_id)` — `CREATE SCHEMA t_{tenant_id}` in RDS, run Drizzle migrations for user app tables
- `create_s3_folder(tenant_id)` — `boto3 put_object` creates `{tenant_id}/` prefix in S3
- `store_secret(tenant_id, key, value)` — puts secret at `onenexium/tenants/{tenant_id}/{key}`
- `get_secret(tenant_id, key)` — retrieves secret from Secrets Manager
- Test `provision_tenant_schema`: creates schema `t_test001` in RDS with contacts table
- Test `store_secret` and `get_secret` round-trip: store a value → retrieve it → values match

T3.4 MCP /deploy — 4 tools ⌚ 2 hrs 👤 Engineer 1

- Claude Code prompt: "In /deploy FastMCP namespace, implement:",
- `build_docker_image(project_id, image_tag)` — build image from `/data/projects/{id}`, push to ECR
- `start_container(project_id, image_uri, subdomain, port)` — SSH to Runtime EC2, docker run with Traefik labels
- `register_traefik_route(project_id, subdomain, port)` — verify route is live by polling the URL
- `register_custom_domain(project_id, domain)` — add Traefik routing rule for custom domain
- paramiko SSHClient for Runtime EC2 SSH (use internal VPC IP, key stored in Secrets Manager)
- Test full chain manually via curl: write files → build → docker push → start container → URL live

✓ END OF DAY ACCEPTANCE CRITERIA

1. All 11 MCP tools respond correctly to curl requests
2. `trigger_build` returns `status:success` for valid Next.js code
3. Self-healing loop calls Claude API and auto-fixes a broken TypeScript file
4. `provision_tenant_schema` creates a real PostgreSQL schema in RDS
5. Manual deploy test: test page renders at <https://test.sites.onenexium.com>

🎯 MILESTONE: All 11 core MCP tools built and manually tested.

DAY
4

Chat UI + Claude Integration + Live Preview

Connect Claude to all MCP servers. Build the chat UI. Wire streaming. Add live preview iframe.

T4.1 AI Generation API Route ⌚ 2 hrs 👤 Engineer 2

- Create `app/api/ai/generate/route.ts`
- Claude Code prompt: "Build a Next.js API route that: (1) loads project context from Drizzle, (2) assembles system prompt with project details and constraints, (3) calls Claude API `claude-sonnet-4-6` with `mcp_servers` array pointing to all 5 MCP namespaces, (4) streams the response as SSE back to the client, (5) parses `tool_use` events and emits status updates as separate SSE events."
- MCP servers: `workspace`, `codegen`, `build`, `infra`, `deploy` — all at `https://mcp.onenexium.com/{namespace}`
- Authorization: `MCP_AUTH_TOKEN` from environment per request
- Error handling: if Claude API fails → SSE error event → client shows retry button

⚡ **Claude Code / Cursor:** Key SSE pattern: use `ReadableStream` + `TransformStream` to pipe Claude streaming response to client. Claude Code knows this pattern well. Specify: "use the Web Streams API, not Node.js streams."

T4.2 Chat Interface — Full Build ⌚ 3 hrs 👤 Engineer 2

- Claude Code prompt: "Build the OneNexium chat interface component at `app/(app)/projects/{id}/page.tsx`:",
- Layout: 2-column — chat (left, 40%) + preview iframe (right, 60%)

- Chat: message history (user right/indigo, AI left/surface), streaming text animation
- Tool status chips: small rounded badges showing current operation
- Input bar: fixed bottom, auto-resize textarea, send on Enter (Shift+Enter = newline), file upload icon
- Preview pane: iframe with sandbox="allow-scripts allow-same-origin", refreshes when URL updates
- Mobile: stacked vertically (chat top, preview below), preview collapsible
- Wire to /api/ai/generate — send message → stream response → parse tool events → update UI
- Test: send "hello" → streaming response appears

⚡ **Claude Code / Cursor:** *Cursor is excellent for the streaming UI — it can see the code and iterate quickly. After Claude Code generates the skeleton, use Cursor to refine the message bubble animations.*

T4.3 System Prompt — Code Generation v1 ⌚ 1.5 hrs 👤 Both Engineers

- Write the system prompt in lib/ai/system-prompt.ts
- Include: Feature 3 spec (multi-page file list), Feature 4 spec (backend generation)
- Key instructions: file order, mobile-responsive requirement, shadcn/ui only, Drizzle for DB
- Add: "After all files are written and trigger_build returns success, immediately call build_docker_image, then start_container, then register_traefik_route. Do not stop until the URL is returned."
- Context function: assembleContext(project) → injects business name, industry, brand colour, current pages
- Test: type "build a landing page for a bakery called Golden Crust" → confirm Claude calls write_file

T4.4 Projects List + Create Project Flow ⌚ 1.5 hrs 👤 Engineer 2

- Create app/(app)/projects/page.tsx — grid of project cards
- Each card: project name, status badge, subdomain link, "Open" button
- Empty state: illustration + "Create your first project" CTA
- Create new project modal: project name input → creates record in RDS → redirects to chat
- Project card on click: goes to /projects/{id} (the chat page)
- Store last_visited_at — show recently active projects first
- Persist conversation: ai_conversations table stores messages as JSON array
- On page load: load conversation history → prepopulate chat UI

✅ END OF DAY ACCEPTANCE CRITERIA

1. Sending a message in chat triggers Claude API call (visible in Network tab)
2. Claude calls write_file and files appear in /data/projects/{id}/ on disk
3. Tool use status chips appear in real time during generation
4. After successful build, deploy chain triggers and URL appears in chat
5. Preview iframe loads the generated site
6. Conversation history persists across page reloads

🎯 **MILESTONE: End-to-end pipe live. Type → Generate → Deploy → Preview — all in one screen.**

PHASE 3 DAYS 5 – 6

Multi-Page + Backend + Beautiful Generated Output

DAY
5

Multi-Page Generation + Backend Code Generation

Claude generates all 5 pages + Drizzle schema + API routes in one shot

T5.1 Multi-page system prompt iteration ⌚ 3 hrs 👤 Both Engineers

- Test multi-page generation: "Build a complete website for a life coaching business called Elevate"
- Target: Claude writes app/layout.tsx, page.tsx, about/page.tsx, services/page.tsx, contact/page.tsx, components/Navbar.tsx, components/Footer.tsx
- Common failures to fix: layout.tsx missing, Navbar not imported, incorrect relative imports
- System prompt fix for each failure — add specific rule
- Success criteria: all 7 files written, build passes on first or second attempt
- Run 5 different business prompts — achieve 4/5 writing all required pages

⚡ **Claude Code / Cursor:** Use Cursor to compare generated output across multiple runs. "What is different between the successful and failing generations?" helps identify the prompt fix needed.

T5.2 Backend generation phase — detect + generate ⌚ 3 hrs 👤 Engineer 1

- Add to system prompt: "After writing all frontend files and trigger_build returns success, scan the frontend for forms and interactive features.",
- Contact form found → generate lib/db/schema/contacts.ts + app/api/contact/route.ts
- Newsletter input found → generate lib/db/schema/subscribers.ts + app/api/subscribe/route.ts
- Booking form found → generate lib/db/schema/bookings.ts + app/api/bookings/route.ts
- Each generated API route: Zod validation, Drizzle insert, Resend email notification
- After schema generation: AI calls provision_tenant_schema to run migrations on RDS
- Test: "Build a coaching website with a contact form and email newsletter" → contact.ts + route.ts generated

T5.3 Generated app UI quality — system prompt v2 ⌚ 2 hrs 👤 Both Engineers

- Add few-shot examples to system prompt showing high-quality component patterns:
- Hero section: full-width, gradient background, heading + sub + CTA button + image
- Services section: 3-column card grid with icons, titles, descriptions
- Testimonials: quote cards with avatar, name, role
- Contact: split layout — info left, form right
- Add: "Always use the brand colour (CSS var --primary) for CTAs, headings accents, and active elements"
- Add: "Every section must have proper padding (py-16 md:py-24) and max-width container"
- Run 5 tests — compare quality to Day 4 output — quality should visibly improve

✓ END OF DAY ACCEPTANCE CRITERIA

1. 5-page site (layout + 4 pages + Navbar + Footer) generates and builds successfully
2. Contact form in generated site posts to /api/contact and saves to RDS
3. Generated app UI has proper sections, spacing, and uses the brand colour
4. Backend migrations run on tenant schema in RDS
5. 4/5 test prompts result in successful multi-page generation

🎯 **MILESTONE: Multi-page + backend generation working. Quality is good enough to show externally.**

DAY

6

Custom Domain + RBAC + Team Invites

Domain connection flow, role-based UI, invite team members

T6.1 Custom Domain Connection Flow ⌚ 3 hrs 👤 Engineer 1

- Generate with Claude Code using Feature 6 spec from Section 03
- Claude Code generates: `app/(app)/projects/{id}/domain/page.tsx` + `app/api/domains/verify/route.ts`
- Drizzle schema: domains table with status enum (pending, verifying, verified, failed)
- DNS polling: server-side cron every 60s via Node.js dns module — update DB on match
- UI states: Enter domain → Showing DNS instructions → Polling animation → Verified → SSL active
- On verify: call `/deploy/register_custom_domain` MCP tool → Traefik adds rule → cert issued
- Test: add a subdomain of a domain you control → verify the full flow end-to-end

⚡ **Claude Code / Cursor:** Claude Code generates the polling mechanism cleanly. Specify: "use `setInterval` on the server side stored in a Map keyed by domain, clean up after verify or after 48hr timeout."

T6.2 RBAC — Role-Based UI ⌚ 2 hrs 👤 Engineer 2

- Implement RBAC using `lib/auth/rbac.ts` generated on Day 1
- Admin: sees all projects, team management, billing section, delete buttons
- Member: sees own projects, can create/edit, cannot manage team or see billing
- Viewer: read-only — chat input is disabled, no create buttons, can see deployed URLs
- Apply role guards to: delete project button, "Invite Team" menu item, settings page, domain settings
- Visual indicator: role badge in sidebar under user avatar
- Test: create two users with different roles — confirm UI differences

T6.3 Team Invites — Basic Flow ⌚ 2 hrs 👤 Engineer 2

- Create `app/(app)/settings/team/page.tsx`
- Claude Code prompt: "Build a team management page: show existing team members (name, email, role, joined date), invite by email input, send invite via Resend with magic link, accept invite creates user account with member role."
- Drizzle schema: `team_invites`: `id`, `workspace_id`, `email`, `role`, `token`, `status`, `created_at`

- Accept invite: GET /api/invites/accept?token={token} → creates user → assigns to workspace
- Test: invite an email → accept via link → user appears in team list with member role

T6.4 Domains Dashboard UI 1 hr Engineer 2

- Create app/(app)/domains/page.tsx — list all domains across all projects
- Status badges: Pending (yellow), Verifying (blue spinner), Verified (green), SSL Active (green with lock)
- Each domain row: domain, project name, status, verified date, delete button
- DNS instructions modal: opens on click of pending domain — shows CNAME record details
- Mobile responsive table — collapses to card view on small screens

✓ END OF DAY ACCEPTANCE CRITERIA

1. User enters a domain → sees DNS instructions → platform polls → verifies → SSL active
2. Admin user sees delete buttons and team management. Viewer user does not.
3. Team invite email is sent → acceptance link works → invited user can log in
4. Domains page shows all domains with correct status badges
5. Domains page is mobile responsive (tested at 375px width)

 **MILESTONE: Custom domains, RBAC, and team invites — all working.**

PHASE 4 DAYS 7 – 8
Quality + Polish + Stability

DAY
7

UI Polish + Mobile Responsiveness + System Prompt v3

Make everything visually excellent. Test on all screen sizes. Run 20 prompts.

T7.1 Platform-wide UI polish pass ⌚ 4 hrs 👤 Engineer 2

- Cursor prompt on each page: "Review this page for visual quality and apply these improvements: consistent spacing (use Tailwind gap-4, p-6 consistently), add hover states to all interactive elements, ensure all buttons use the primary colour scheme, add loading skeletons where data is fetched, fix any text overflow issues."
- Pages to polish: login, onboarding, /app dashboard, projects list, chat interface, domains, settings
- Add: page transition animations (Framer Motion — fade in on route change)
- Add: empty states with illustrations for: no projects, no domains, no team members
- Add: proper 404 and error pages
- Mobile test every page at 375px, 768px, 1280px — fix any breakage

⚡ **Claude Code / Cursor:** *Cursor shines here. Open a page, select a section, and ask "improve the visual design of this component" — it iterates quickly on UI details. Do this page by page.*

T7.2 Generated app quality — 20 prompt test ⌚ 3 hrs 👤 Both Engineers

- Run 20 diverse prompts (different industries, lengths, specific requirements)
- Record for each: pages generated, build pass/fail, quality score (1-5), deploy success
- Target: 80% build success, average quality 3/5 or above
- For each failure: identify root cause (missing file, bad import, TS error type)
- Update system prompt v3: fix top 5 failure patterns
- Re-run the failed prompts after fix — confirm improvement
- Document the 3 best-looking generated sites — screenshot and add to SHOWCASE.md

T7.3 Self-healing loop visual feedback ⌚ 1 hr 👤 Engineer 2

- When self-healing retry triggers: show in chat UI:
- "Build failed — fixing 3 TypeScript errors automatically... (attempt 2/3)"
- Show error list as collapsed expandable section (user can see what was wrong)
- On success after retry: "Fixed and rebuilt successfully ✓"
- On all retries exhausted: "Could not auto-fix. Try describing the change differently."
- Add a "Try again" button when all retries fail

✅ END OF DAY ACCEPTANCE CRITERIA

1. All platform pages look visually polished on desktop and mobile
2. No layout breaks at 375px (iPhone SE) on any page

3. 80% or more of 20 test prompts result in successful builds
4. Self-healing loop shows proper status messages during retry
5. Empty states exist for all list pages

 **MILESTONE: 80% build success. Beautiful platform UI. Mobile responsive throughout.**

DAY
8

Settings + Notifications + Integration Testing

Complete settings pages, add notifications, full end-to-end integration test

T8.1 Settings pages — workspace + account 2 hrs Engineer 2

- Claude Code: "Build workspace settings page at /settings: workspace name, description, brand colour (colour picker), logo upload (S3), delete workspace (with confirmation modal + type-to-confirm). Build account settings: display name, email, avatar, change password, sign out all sessions."
- Apply changes save to Drizzle immediately (no page reload required)
- Delete workspace: soft delete (status=deleted), cancel all deployments, send confirmation email
- Logo upload: presigned S3 URL → client uploads directly → update workspace.logo_url
- Test: change brand colour → re-generate a site → confirm new colour used

T8.2 In-app notification system 2 hrs Engineer 2

- Claude Code: "Build an in-app notification system: bell icon in top bar with unread count badge, dropdown showing last 10 notifications, mark as read on click."
- Drizzle schema: notifications: id, user_id, type, message, data (jsonb), read, created_at
- Trigger notifications: deployment successful, domain verified, team member joined, build failed after retries
- Real-time: use polling (GET /api/notifications every 30s) — WebSocket in Sprint 1
- Test: complete a deployment → notification appears in bell → click → marks as read

T8.3 Full integration test — both engineers 3 hrs Both Engineers

- Run the complete user journey 5 times from scratch (incognito browser, no existing data):
 1. Sign up → Google OAuth → Onboarding (4 steps) → /app
 2. Create new project → type prompt → multi-page site generated → deployed
 3. Add another message → "Add a booking section" → backend generated
 4. Connect custom domain (use a test subdomain you own) → verified → SSL active
 5. Invite team member with viewer role → they sign up → see project (read-only)
- Document every bug found → fix High severity immediately
- Run on mobile (375px) → confirm every step works on mobile
- Screenshot every screen of the journey → add to DEMO.md

✓ **END OF DAY ACCEPTANCE CRITERIA**

1. All settings changes save and persist correctly
2. Notification appears after deployment completes
3. Full user journey (signup → onboard → build → deploy → domain → invite) works 4/5 times
4. Full journey works on mobile (375px)
5. All High severity bugs fixed



MILESTONE: Full feature set works end-to-end. Platform ready for external users.

PHASE 5 **DAYS 9 – 10**
Harden + Demo + Show the World

DAY
9

Final Hardening + 25-Prompt Test + Demo Prep

Fix remaining bugs. Run comprehensive prompt test. Prepare demo script.

T9.1 Bug fix sprint — all High severity issues ⌚ 3 hrs 👤 Both Engineers

- Work through BUGS.md — fix every item marked High
- Automated: write a simple test script (test-flow.sh) that calls each MCP tool and confirms responses
- Load test: deploy 5 different projects simultaneously — confirm no port conflicts, all containers healthy
- Disk usage: confirm cleanup cron is running — check /data usage: df -h
- RDS connections: confirm all containers can connect — check for connection pool exhaustion
- SSL: confirm all deployed sites have valid SSL — check certificate expiry dates in Traefik

T9.2 25 prompt comprehensive test — coaching vertical focus ⌚ 2 hrs 👤 Both Engineers

- Run 25 prompts specifically from the coaching/consulting vertical:
- Life coaches, business coaches, fitness coaches, therapists, online course creators,
- consultants, freelancers, marketing agencies, photographers, personal trainers
- For each: record build success, quality score (1-5), any failures
- Target metrics: 85% build success, 90% deploy success, average quality 3.5/5
- Fix system prompt for any new failure patterns found
- Select the 3 best-quality generated sites for the demo

T9.3 Demo script + environment prep ⌚ 1.5 hrs 👤 Both Engineers

- Write DEMO.md with exact script for Day 10
- Pre-create two demo accounts: demo-coach@onenexium.com and demo-client@onenexium.com
- Pre-run the demo prompt once — screenshot the result — have it ready as backup
- Prepare 3 pre-tested prompts that always produce excellent output
- Write FAQ answers: "How does it work?", "Is the code real?", "Can I use my own domain?", "How much will it cost?", "What can't it do yet?"
- Practice the demo twice — time it (target: 8 minutes from "I'll sign up" to "domain is live")

T9.4 Documentation + README ⌚ 1.5 hrs 👤 Both Engineers

- README.md (onenexium-platform): what it is, how to run locally, env vars, deploy
- README.md (onenexium-mcp): what it is, setup, tool reference, curl examples
- WHAT_WORKS.md: honest list of working features
- KNOWN_ISSUES.md: honest list of limitations and bugs
- SPRINT_1_BACKLOG.md: everything deferred from Sprint Zero
- Commit everything to GitHub — clean history with descriptive commit messages

✓ END OF DAY ACCEPTANCE CRITERIA

1. 85% build success on 25-prompt coaching vertical test
2. All 5 simultaneous deployments succeed without conflicts
3. Demo script written and practiced twice
4. All documentation committed to GitHub
5. onenexium-platform and onenexium-mcp repos have complete READMEs

 **MILESTONE: System is hardened. Demo is rehearsed. Ready for Day 10.**

DAY

10

Demo Day — Show 3 Real People the Working Product

External demo, collect feedback, define Sprint 1, celebrate

T10.1 Morning system check (1 hour before demo) ⌚ 1 hr 👤 Both Engineers

- Verify all systems running: Platform EC2, Runtime EC2, RDS, MCP server
- Run the complete demo prompt in incognito — confirm it works
- Check disk space on both EC2s: `df -h` — at least 50% free
- Check active containers: `docker ps` on Runtime EC2 — all healthy
- Check MCP server: `curl https://mcp.onenexium.com/health → 200`
- Check Platform: navigate to `onenexium.com` → login → onboarding → chat — all load
- Fallback plan: if anything breaks, use the pre-generated screenshot backup

T10.2 The Demo — 3 External People ⌚ 2 hrs 👤 Founder + Both Engineers

- Show the product to 3 people outside the team:
- Ideal: potential users from the coaching vertical
- Alternative: developer friends, advisors, investors
- Run demo script from DEMO.md:
- Part 1 (4 min): Sign up, onboard, generate 5-page coaching site, show live URL
- Part 2 (2 min): Add a contact form, watch backend generate, show API route created
- Part 3 (2 min): Connect domain (use pre-configured test domain for speed)
- Let each person try it themselves — watch where they get stuck or confused
- Record reactions specifically: what surprised them, what frustrated, would they pay?
- Collect: business type, email, willingness to pay estimate

T10.3 Sprint Zero Retrospective ⌚ 45 min 👤 Both Engineers

- Answer and document in RETRO.md:
- What did Claude Code save the most time on?
- What feature took longer than expected and why?
- What was the hardest technical challenge?
- What would we do differently if starting tomorrow?

- What is the single biggest risk for Sprint 1?
- Commit RETRO.md to GitHub

T10.4 Define Sprint 1 — 2 Weeks Starting Day 11 1 hr Both Engineers + Founder

- Based on demo feedback, define the Sprint 1 backlog:
- P0 — Visual overlay editor (click-to-edit any component)
- P0 — Billing (Stripe — Free/Pro/Business plans)
- P0 — Generation quality improvements (prompt v4 based on test data)
- P0 — Real-time build status via WebSocket (replace polling)
- P1 — Coaching vertical templates (5 pre-built starting points)
- P1 — More backend generation (Stripe checkout, auth in generated app)
- P1 — Performance: reduce generation time from 3 min to 90 seconds
- P2 — Advanced RBAC (workspace-level permissions audit)
- Create Sprint 1 issues in GitHub Projects — assign owners

END OF DAY ACCEPTANCE CRITERIA

1. Demo runs successfully in front of 3 external people without crashing
2. At least 1 external person generates their own site during the demo
3. Retrospective written and committed
4. Sprint 1 backlog defined with P0/P1/P2 priorities and owner assignments
5. At least 1 person from the demo asks "when can I sign up for real?"

 **MILESTONE: SPRINT ZERO COMPLETE. Full-stack AI web app builder — working, beautiful, and shown to real users.**

05 RISKS & MITIGATION

Everything that can slow or kill the 10-day timeline

RISK REGISTER

Risk	Level	Mitigation
Claude Code generates wrong patterns for Next.js 14 App Router	High	Always specify "Next.js 14 App Router, not pages router" in every Claude Code prompt. If generated code uses pages/, reject and re-prompt.
Anthropic MCP endpoint unreachable from Claude API	High	Test public reachability of mcp.onenexium.com from outside your network (use reqbin.com) before Day 5. Keep ngrok as backup.
Scope creep from expanded feature list	High	The founder reviews TASKS.md every morning. Any task not on the day's list goes to BACKLOG.md immediately.
Drizzle migration conflicts between platform and tenant schemas	Medium	Platform schema uses search_path=public. Tenant schemas use t_{tenant_id}. Keep them completely separate. Never mix.
Let's Encrypt rate limit (50 certs/week per domain)	Medium	Use staging certificate resolver during development (Days 1-8). Switch to production resolver only for Day 9-10 real demos.
NextAuth v5 breaking changes from v4 patterns	Medium	Specify "NextAuth v5 (Auth.js)" explicitly in every prompt. Claude Code sometimes generates v4 patterns. Check: auth.ts uses handlers export, not nextAuthOptions.
Self-healing loop costs too much in Claude API tokens	Medium	Cap retries at 3. Each retry call is ~2,000 tokens. Monitor daily spend. If > \$50/day: reduce max retries to 2.
Cursor / Claude Code generates code that compiles but is wrong	Low	Test the functionality, not just the compilation. After every generated feature: manually use it as a user would.
EC2 disk full from Docker image layers	Medium	docker system prune -f daily. Store images in ECR not locally. Monitor: df -h every morning.
One engineer significantly slower than estimated	Medium	Claude Code acceleration is assumed throughout. If a task takes 2x the estimate, reprioritise rather than crunch. Drop a lower-priority feature.

06 AFTER DAY 10 — THE ROAD AHEAD

Sprint 1 priorities, quality roadmap, and path to first paying users

SPRINT 1 — DAYS 11–24

Priority	Feature	Target Outcome
P0	Visual overlay editor — click-to-edit any component	Users can refine generated sites without re-prompting from scratch
P0	Billing — Stripe Free/Pro/Business	Revenue from day one. Gate features behind plan limits.
P0	Generation quality v4 — trained on Sprint Zero data	Build success rate: 85% → 95%. Average quality: 3.5 → 4.2/5.
P0	Real-time build status via WebSocket	Replace 30s polling. Instant feedback during generation.
P1	Coaching templates — 5 pre-built starting points	Reduce time-to-first-site from 3 min to 45 seconds for common use cases
P1	Stripe payment generation in user apps	Generate Stripe Checkout in user apps — key feature for coaches
P1	Performance — generation time 3 min → 90 seconds	Semantic caching + parallel file writes + faster build sandbox
P2	SEO auto-config — meta tags, sitemap, OG images	Every generated site is search-engine ready out of the box

THE ROADMAP AFTER SPRINT ZERO

Day 10 — Sprint Zero complete:

- ✓ Auth + RBAC + Onboarding
- ✓ Multi-page generation
- ✓ Backend generation (Drizzle + API routes)
- ✓ Self-healing build loop
- ✓ Custom domain + SSL
- ✓ Beautiful UI + Mobile responsive

Quality: good enough to demo. Not good enough to charge for.

Month 1 (Sprint 1 + 2):

Visual editor, billing, quality 95%, coaching templates
First 20 paying users

Month 2 (Sprint 3 + 4):

More backend types (auth in generated app, Stripe in generated app)
Monitoring, Langfuse observability, analytics dashboard
First 100 paying users

Month 3 (Sprint 5 + 6):

Public beta — coaching vertical launch
Agency white-label program
First \$10k MRR

Month 6: 200 users. \$25k MRR.
Month 12: 1,000 users. \$100k MRR. Series A ready.

"The product is built. The architecture is proven. The only variable now is how fast you talk to customers and iterate based on what they tell you."

OneNexium · Sprint Zero Revised · Claude Code + Cursor Accelerated · Version 2.0

This document supersedes the v1 Sprint Zero plan. Expanded scope is enabled by AI-assisted development.